

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ»

КАФЕДРА «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 3
ПО ДИСЦИПЛИНЕ «ТИПЫ И СТРУКТУРЫ ДАННЫХ»

Обработка разреженных матриц

Вариант № 5

Студент: **Чернышева А.П.**

Группа: **ИУ7-36Б**

Преподаватель: **Силантьева А. В.**

2025 г.

Описание условия задачи

Первая разреженная (содержащая много нулей) матрица хранится в форме 3-х объектов (CSR):

- вектор A содержит значения ненулевых элементов;
- вектор IA содержит номера столбцов для элементов вектора A;
- вектор JA, в элементе Nk которого находится номер компонент в A и IA, с которых начинается описание строки Nk матрицы A.

Вторая разреженная матрица хранится в форме 3-х объектов (CSC):

- вектор B содержит значения ненулевых элементов;
- вектор JB содержит номера строк для элементов вектора A;
- вектор IB, в элементе Nk которого находится номер компонент в B и JB, с которых начинается описание столбцов Nk матрицы B.

Смоделировать операцию умножения двух матриц:

1. Хранящихся в указанной форме применяя алгоритм работы со сжатыми матрицами, с получением результата в форме хранения первой матрицы (CSR).
2. Хранящиеся в стандартной форме, применяя стандартный алгоритм работы с матрицами.
3. Сравнить время выполнения операций и объем памяти при использовании этих 2-х алгоритмов при различном проценте заполнения матриц.

Описание ТЗ

Описание исходных данных

Введенный пункт меню.

Размеры матриц, элементы.

Ограничения:

— Максимальный размер матрицы: 1000 × 1000 элементов;

— Количество элементов должно быть целым положительным числом;

— Поддерживаются целочисленные значения элементов.

Формат ввода: обычный (все элементы) или координатный (только ненулевые).

Ограничения при координатном вводе:

— Матрица должна содержать хотя бы один ненулевой элемент;

— Индексация начинается с единицы;

— Индекс не может превышать размеры матрицы;

— Индекс является положительным целым числом.

Результаты:

Умноженная матрица в выбранном формате.

Анализ эффективности.

Сгенерированные матрицы.

Выведенные матрицы в обычном или координатном формате.

Описание задачи, реализуемой программой

Программа реализует систему работы с разреженными матрицами с возможностью:

1. Ввод матриц: ручной ввод матриц с выбором формата (обычный или координатный)
2. Генерация матриц: создание случайных матриц с заданными параметрами

3. Умножение матриц: два варианта умножения - стандартный и через разреженные форматы
4. Отображение матриц: вывод в обычном и разреженном форматах
5. Анализ производительности.

Способ обращения к программе

Программа запускается через исполняемый файл app.exe.

Описание возможных аварийных ситуаций и ошибок пользователя

1. Ошибка ввода данных (INPUT_ERROR)
2. Ошибка выделения памяти (INPUT_ERROR)
3. Ошибка операций с матрицами (MULT_ERROR)
4. Ошибка работы с пустыми данными
5. Ошибка переполнения
6. Ошибка формата данных

Описание внутренних структур данных

CompactCSR для формата CSR:

```
struct CompactCSR {
    int *elements;
    int *column_indexes;
    int *row_starts;
    int num_rows;
    int num_cols;
    int element_count;
    int buffer_size;
};
```

elements - хранит значения ненулевых элементов матрицы
column_indexes - содержит номера столбцов для каждого элемента из elements
row_starts - указывает позиции в elements и column_indexes, где начинается описание каждой строки
element_count - текущее количество хранимых ненулевых элементов
buffer_size - максимальная вместимость массивов

CompactCSC для формата CSC:

```
struct CompactCSC {
    int *elements;
    int *row_indexes;
    int *col_starts;
    int num_rows;
    int num_cols;
    int element_count;
    int buffer_size;
};
```

elements - хранит значения ненулевых элементов матрицы
row_indexes - содержит номера строк для каждого элемента из elements
col_starts - указывает позиции в elements и row_indexes, где начинается описание каждого столбца

element_count - текущее количество хранимых ненулевых элементов
buffer_size - максимальная вместимость массивов
StandardMat для хранения матрицы в стандартном виде

```
struct StandardMat {
    int **matrix;
    int num_rows;
    int num_cols;
};
```

matrix - двумерный массив для хранения всех элементов матрицы (включая нули)
num_rows, num_cols - размеры матрицы

Общий алгоритм программы:

1. Инициализация программы:
    1. Объявление структур для хранения матриц в различных форматах
    2. Инициализация переменных для управления памятью и состоянием программы

2. Основной цикл программы:
    1. Вывод меню с доступными операциями
    2. Ожидание ввода выбора пользователя
    3. Выполнение выбранной операции

3. Операции меню
    1 - Ввод матриц вручную
    2 - Генерация случайных матриц
    3 - Умножение матриц (стандартный формат)
    4 - Умножение матриц (CSR * CSC формат)
    5 - Отображение матриц (стандартный формат)
    6 - Отображение матриц (разреженный формат)
    7 – Анализ эффективности
    0 - Выход из программы

Описание алгоритмов функций

int read_standard_matrix(struct StandardMat *mat, int fixed_rows)- основной ввод матрицы:

1. Запрос размеров матрицы с валидацией
2. Выбор способа ввода (обычный/координатный)
3. Выделение памяти и заполнение данных
4. Преобразование в разреженные форматы

int read_coordinate_matrix(struct StandardMat *mat)- координатный ввод:

1. Цикличный ввод троек (строка, столбец, значение)
2. Валидация координат и значений
3. Проверка на наличие хотя бы одного ненулевого элемента
4. Завершение ввода по специальному символу

int create_random_matrix(struct StandardMat *mat, int rows, int cols, int non_zero_count) - генерация случайной матрицы:
1. Запрос параметров (размеры, количество ненулевых)
2. Случайное распределение ненулевых элементов
3. Гарантия уникальности позиций ненулевых элементов

int convert_standard_to_csr(const struct StandardMat *std_mat, struct CompactCSR *csr) - преобразование в CSR:
1. Подсчет ненулевых элементов
2. Инициализация структур CSR
3. Последовательное добавление ненулевых элементов
4. Построение корректных указателей строк

int convert_standard_to_csc(const struct StandardMat *std_mat, struct CompactCSC *csc) - преобразование в CSC:
1. Подсчет ненулевых элементов по столбцам
2. Инициализация структур CSC
3. Заполнение данных с сортировкой по столбцам
4. Построение корректных указателей столбцов

int convert_csr_to_standard(const struct CompactCSR *csr, struct StandardMat *std_mat) - обратное преобразование:
1. Выделение памяти для плотной матрицы
2. Инициализация нулями
3. Заполнение ненулевых элементов по данным CSR

int multiply_standard_matrices(const struct StandardMat *a, const struct StandardMat *b, struct StandardMat *result) - стандартное умножение:
1. Проверка совместимости размеров
2. Тройной вложенный цикл для вычисления произведения
3. Поэлементное накопление результатов

int multiply_csr_csc_matrices(const struct CompactCSR *a, const struct CompactCSC *b, struct CompactCSR *res) - разреженное умножение:
1.Инициализация результирующей матрицы
2.Построчное вычисление произведений
Для каждой строки i матрицы a пределяет диапазон ненулевых элементов в строке через a->row_starts[i] и a-> row_starts[i+1]
Для каждого столбца j матрицы b определяет диапазон ненулевых элементов в столбце через b->col_starts[j] и b-> col_starts[j+1]
Использует два указателя (a_pos и b_pos) для одновременного обхода:
Ненулевых элементов строки i матрицы a
Ненулевых элементов столбца j матрицы b
Сравнивает индексы столбцов a_col и строк b_row:
Если a_col < b_row - увеличивает указатель a_pos
Если a_col > b_row - увеличивает указатель b_pos
При равенстве - вычисляет произведение и добавляет к сумме
3. Оптимизация памяти

void display_standard_matrix(const struct StandardMat *mat) - вывод плотной матрицы:
1. Проверка размера для отображения
2. Форматированный вывод в виде таблицы
3. Автоматическое определение ширины столбцов

void display_csr_format(const struct CompactCSR *csr) и void display_csc_format(const struct CompactCSC *csc)
1. Вывод трех массивов разреженного формата
2. Четкое визуальное разделение компонентов
3. Информативные заголовки для каждого массива

int initialize_csr(struct CompactCSR *csr, int rows, int cols, int capacity) и int initialize_csc_format(struct CompactCSC *csc, int rows, int cols, int capacity)
1. Выделение памяти с проверкой успешности
2. Инициализация всех полей структуры
3. Установка начальной емкости

int insert_element_csr(struct CompactCSR *csr, int row, int col, int val) - добавление в разреженную матрицу:
1. Проверка необходимости расширения capacity
2. Добавление элемента и обновление метаданных
3. Поддержка целостности структуры данных

void run_performance_test(void) - комплексное тестирование:
1. Многократное выполнение операций для статистики
2. Замер времени с высокой точностью
3. Расчет использования памяти для разных форматов
4. Сравнительный анализ эффективности

size_t calculate_standard_memory(const struct StandardMat *mat) и size_t calculate_csr_memory(const struct CompactCSR *mat)
1. Точный расчет занимаемой памяти
2. Учет всех компонентов структур данных
3. Сравнение для разных уровней заполнения**:**

void clean_input_buffer(void) - очистка буфера ввода

void show_menu(void) - управление основным циклом программы

int create_random_matrix(struct StandardMat *mat, int rows, int cols, int non_zero_count) – генерация рандомной матрицы
1. Валидация входных параметров
2. Инициализация структуры матрицы
3. Создание нулевой матрицы
4. Генерация случайных ненулевых элементов

Тесты

Позитивные тесты

Номер теста	Описание	Ожидаемый результат
1	Ввод двух матриц построчно 1 матрица	Успешная запись матриц

	2 2 1 2 3 4 2 матрица 2 2 1 2 3 4	
2	Перемножение матриц обычным способом из теста 1	Умноженная матрица 7 10 15 22
3	Ввод двух матриц покоординатно 1 матрица 2 2 0 0 1 0 1 2 2 матрица 2 2 0 1 1 1 1 4	Успешное считывание данных из файла и вывод на экран.
4	Умножение разреженных матриц(матрицы из теста 3)	Элементы (A): 9 Начала строк (IA): 0 1 1 Индексы столбцов (JA): 1 Тот же результат в стандартном формате: Полная матрица 2x2: 0 9 0 0
5	Вывод матрицы после умножения	Успешный вывод матрицы.
6	Анализ эффективности	Успешная вывод таблиц сравнения эффективности

Негативные тесты

Номер теста	Описание	Ожидаемый результат

1	Умножение матриц, несовместимых для умножения	Ошибка: количество столбцов первой матрицы должно равняться количеству строк второй матрицы
2	Ввод невалидных данных	Ошибка чтения ввода
3	Ввод координат за пределами матрицы	Ошибка: координаты за пределами матрицы
4	Ввод матрицы без ненулевых элементов при координатном вводе.	Ошибка: матрица должна содержать хотя бы один ненулевой элемент
5	Операция вывода или умножения, если матрицы еще не введены.	Ошибка: матрицы не загружены
6	Ввод размеров матрицы больше MAX_SIZE	Ошибка: неверное количество строк (максимум 1000)
7	Ввод количества ненулевых элементов больше общего числа при координатном вводе	Ошибка: количество ненулевых элементов должно быть в диапазоне
8	Некорректный выбор в меню.	Неверный выбор

Оценка эффективности

Замеры сделаны за 200 итераций.

Размер	Заполнение	Обычные(нс)	Разреженные(нс)	Эффективность(%)
50,00	1.0%	430810.0	6810.0	98.4%
50,00	10.0%	377700.0	148090.0	60.8%
50,00	20.0%	377625.0	349390.0	7.5%
50,00	30.0%	377645.0	548935.0	-45.4%
50,00	40.0%	377670.0	731215.0	-93.6%
50,00	50.0%	377560.0	850515.0	-125.3%
100,00	1.0%	3005860.0	34820.0	98.8%
100,00	10.0%	3010205.0	1387895.0	53.9%
100,00	20.0%	3009525.0	2925140.0	2.8%
100,00	21.0%	3017050.0	3083250.0	-2.2%
100,00	22.0%	3015500.0	3275700.0	-8.6%
100,00	20.5%	3009250.0	3023350.0	-0.5%

100,00	30.0%	3009760.0	4576615.0	-52.1%
100,00	40.0%	3005105.0	6013190.0	-100.1%
100,00	50.0%	3009235.0	6884710.0	-128.8%
200,00	1.0%	23652735.0	764515.0	96.8%
200,00	10.0%	23662985.0	11068770.0	53.2%
200,00	20.0%	23685425.0	23541180.0	0.6%
200,00	30.0%	23678040.0	36814675.0	-55.5%
200,00	40.0%	23690065.0	48373945.0	-104.2%
200,00	50.0%	23654350.0	55243970.0	-133.5%

Для матриц размера 100×100 экспериментально установлена точка равной эффективности при 20.5% заполнении. В этой точке производительность стандартного и разреженного алгоритмов практически идентична.

Размер	Заполнение	Обычные	Разреженные	Эффективность(%)
50	1.0%	10416	444	95.7%
50	10.0%	10416	2244	78.5%
50	20.0%	10416	4244	59.3%
50	30.0%	10416	6244	40.1%
50	40.0%	10416	8244	20.9%
50	50.0%	10416	10244	1.7%
100	1.0%	40816	1244	97.0%
100	10.0%	40816	8444	79.3%
100	20.0%	40816	16444	59.7%
100	30.0%	40816	24444	40.1%
100	40.0%	40816	32444	20.5%
100	50.0%	40816	40444	0.9%
200	1.0%	161616	4044	97.5%
200	10.0%	161616	32844	79.7%
200	20.0%	161616	64844	59.9%
200	30.0%	161616	96844	40.1%
200	40.0%	161616	128844	20.3%
200	50.0%	161616	160844	0.5%

Вывод

Для эффективной работы с разреженными матрицами, содержащими значительное количество нулевых элементов, была реализована система, использующая специализированные форматы хранения данных - CSR и CSC. Это позволило оптимально использовать память, храня только ненулевые элементы и их координаты, вместо полного представления матрицы.

Для матриц размера 100×100 экспериментально установлена точка равной эффективности при 20.5% заполнении. В этой точке производительность стандартного и разреженного алгоритмов практически идентична.

Для матриц размера 200×200 экспериментально установлена точка равной эффективности при 20% заполнении. До этой точки для лучшей производительности эффективнее использовать разряженный алгоритм, после нее – обычный.

По памяти: разреженные форматы эффективны при заполнении менее 50%.

Контрольные вопросы

1. Что такое разреженная матрица, какие схемы хранения таких матриц Вы знаете?

Разреженная матрица - это матрица, в которой большинство элементов равны нулю. Такие матрицы широко используются в научных вычислениях, обработке графиков, машинном обучении и других областях, где данные имеют естественную разреженную структуру.

Существуют различные методы хранения элементов матрицы в памяти. Например, линейный связный список, т.е. последовательность ячеек, связанных в определенном порядке. Существует диагональная схема хранения симметричных матриц, а также - связные схемы разреженного хранения. Связная схема хранения матриц, предложенная Кнутом, предлагает хранить в массиве (например, в AN) в произвольном порядке сами элементы, индексы строк и столбцов соответствующих элементов (например, в массивах I и J), номер (из массива AN) следующего ненулевого элемента, расположенного в матрице по строке (NR) и по столбцу (NC), а также номера элементов, с которых начинается строка (указатели для входа в строку – JR) и номера элементов, с которых начинается столбец (указатели для входа в столбец – JC. Наиболее широко используемая схема хранения разреженных матриц - это схема, предложенная Чангом и Густавсоном, называемая: "разреженный строчный формат". Эта схема предъявляет минимальные требования к памяти и очень удобна при выполнении операций сложения, умножения матриц, перестановок строк и столбцов, транспонирования, решения систем линейных уравнений, при хранении коэффициентов в разреженных матрицах и т.п.

2. Каким образом и сколько памяти выделяется под хранение разреженной и обычной матрицы?

Для хранения разреженной матрицы выделяется память только под ненулевые элементы

Обычная матрица:

- Память: rows × cols × sizeof(int)

Разреженная матрица в формате CSR/CSC:

- Память: (2 × nnz + rows + 1) × sizeof(int), где nnz - количество ненулевых элементов

3. Каков принцип обработки разреженной матрицы?

Основной принцип - исключение операций с нулевыми элементами и минимизация обработки незначащих данных.

4. В каком случае для матриц эффективнее применять стандартные алгоритмы обработки матриц? От чего это зависит?

Стандартные алгоритмы эффективнее при:

1. Высокой плотности заполнения
2. Небольших размерах матриц

Эффективность зависит от:

1. Процент заполнения - главный фактор, определяющий целесообразность
2. Размер матрицы - для больших матриц выгода более значительна